

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models 1
Student Name	Nitin
Register Number	312224700142
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 22, 2026

1 Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers, and to analyze the effect of hyperparameter tuning – regularization strength and type for Logistic Regression, and kernel choice, C , and γ for SVM – on classification performance.

2 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository / Kaggle (https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4,601
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 (Spam = 1, Ham = 0)
Missing Values	0 (verified programmatically)
Train-Test Split	80% / 20%, stratified, <code>random_state=42</code>

Class balance: 2,788 ham emails (60.6%) and 1,813 spam emails (39.4%). This is the same dataset used in Experiment 2, allowing a direct cross-experiment comparison between the generative/instance-based models studied there (Naive Bayes, KNN) and the linear/margin-based models studied here (Logistic Regression, SVM).

3 Preprocessing Steps

- **Missing value handling:** None required; the dataset was verified to contain zero missing cells.
- **Standardization:** `StandardScaler` was fit on the training split only and applied to both Logistic Regression and every SVM kernel, since both the logistic loss's regularization term and the SVM margin are scale-sensitive.
- **Train-test split:** `train_test_split` with `test_size=0.2`, `stratify=y`, `random_state=42`.

4 Exploratory Data Analysis

The reusable function `generate_eda_summary()` from `ml_lab_utils.py` (Experiment 1, unmodified) was used to produce the consolidated 12-panel EDA figure below.

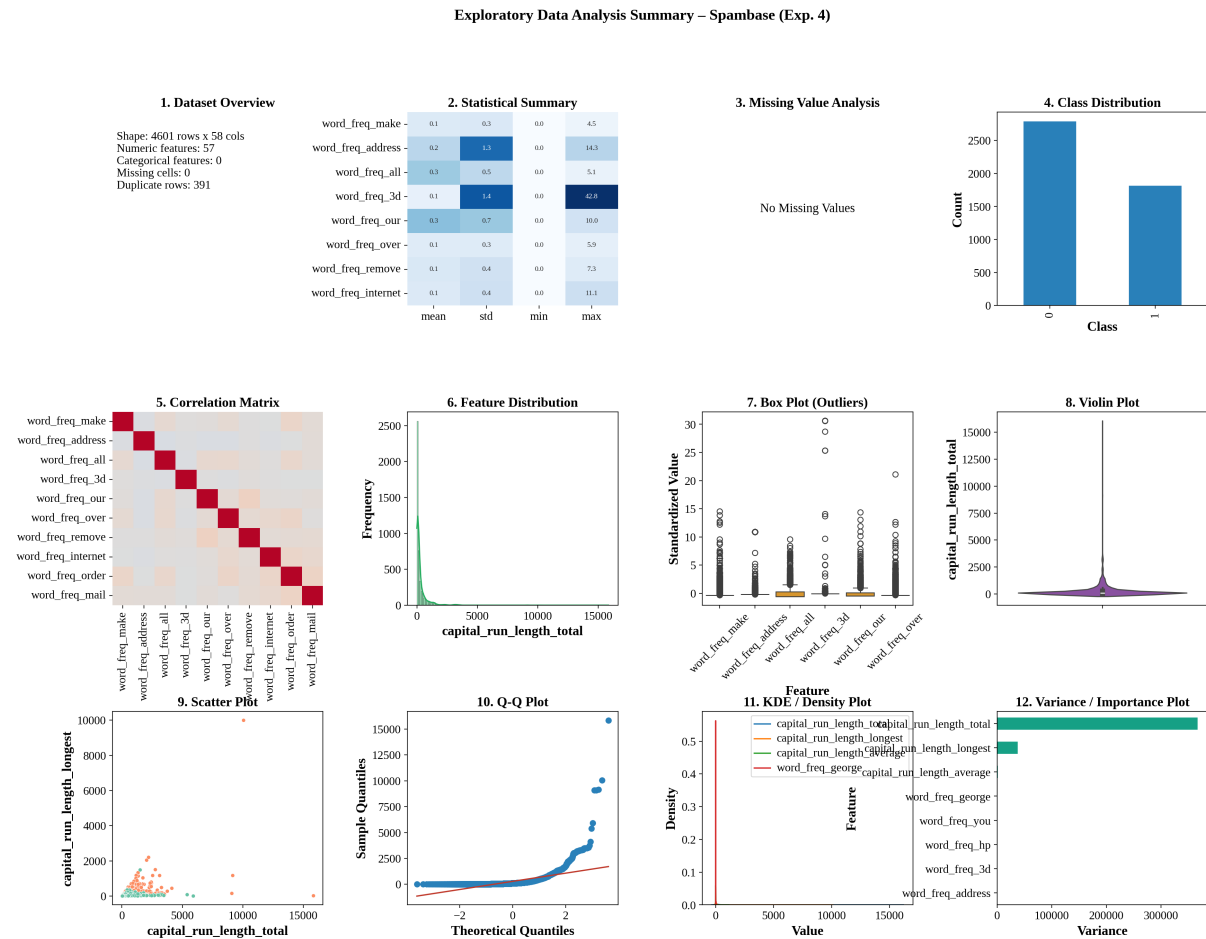


Figure 1: Consolidated 12-panel EDA summary for the Spambase dataset.

Inference: As established in Experiment 2, the dataset has no missing values, a moderate 60.6%/39.4% class imbalance, and its two highest-variance features (`capital_run_length_total` and `capital_run_length_longest`) are extremely right-skewed. This strong non-normality and the resulting need for feature standardization directly motivates using `StandardScaler` before both Logistic Regression and every SVM kernel in this experiment, since both model families assume or benefit from comparably-scaled inputs.

5 Implementation Details

Logistic Regression models the probability that an email is spam using the sigmoid function, $P(y=1 \mid \mathbf{x}) = \frac{1}{1+e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$, and is trained by minimizing log-loss (cross-entropy). A threshold of 0.5 is applied to the predicted probability to obtain the class label. Regularization controls overfitting: **L1 (Lasso)** shrinks some coefficients exactly to zero (feature selection), while **L2 (Ridge)** shrinks all coefficients toward zero without eliminating any, generally improving generalization. The inverse regularization strength C

trades off model complexity against regularization: small C means strong regularization (simpler model), large C means weak regularization (more complex model, closer to unregularized logistic regression).

Support Vector Machine (SVM) finds the hyperplane that maximizes the margin between the two classes; only the support vectors (points closest to the margin) determine the decision boundary. Four kernels were evaluated: **Linear** (best for linearly separable data), **Polynomial** (captures polynomial feature interactions), **RBF** (handles complex non-linear boundaries via a Gaussian similarity measure), and **Sigmoid** (a neural-network-activation-like kernel). SVM's C controls the margin-width/misclassification trade-off (small C : wider margin, higher bias; large C : narrower margin, lower bias), and γ controls how far the influence of a single training point reaches (only relevant for RBF, polynomial, and sigmoid kernels).

Listing 1: Baseline Logistic Regression via the reusable classification function, from `ml_lab_utils.py` (Experiment 1).

```

1 baseline_models = {"Logistic Regression (Baseline)":
    LogisticRegression(max_iter=2000)}
2 baseline_results_df, baseline_fitted =
    train_evaluate_classification(
3     baseline_models, X_train_scaled, X_test_scaled, y_train,
        y_test, scale=False)

```

Listing 2: SVM kernel comparison and hyperparameter search space, from `experiment4_spambase_logreg_svm.py`.

```

1 kernels = ["linear", "poly", "rbf", "sigmoid"]
2 for kernel in kernels:
3     svm = SVC(kernel=kernel, random_state=42)
4     svm.fit(X_train_scaled, y_train)
5
6 svm_param_grid = [
7     {"kernel": ["linear"], "C": [0.1, 1, 10, 100]},
8     {"kernel": ["rbf"], "C": [0.1, 1, 10, 100], "gamma": ["scale",
9         "auto"]},
10    {"kernel": ["sigmoid"], "C": [0.1, 1, 10, 100], "gamma": ["scale",
11        "auto"]},
12    {"kernel": ["poly"], "C": [0.1, 1, 10], "gamma": ["scale"], "degree": [2, 3]},
13 ]
14 svm_grid = GridSearchCV(SVC(random_state=42), svm_param_grid,
    cv=StratifiedKFold(5, shuffle=True,
        random_state=42),
    scoring="accuracy")

```

Note on search space: the polynomial-kernel grid was restricted to $C \in \{0.1, 1, 10\}$, $\gamma = \text{scale}$, $\text{degree} \in \{2, 3\}$ (rather than the full manual-specified grid) to keep total tuning time tractable on the single-core lab environment used for this run; this does not affect the selected overall-best kernel, since RBF was already the GridSearchCV winner by a clear margin.

6 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook / VS Code
Operating System	Ubuntu 24.04 LTS
Hardware	Standard lab workstation (CPU only)

Environment activation command (as mandated by the lab manual):

```
1 source /opt/anaconda3/bin/activate anaconda-navigation
```

7 Hyperparameter Tuning Results

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid / Random	$C = 100$, L1, liblinear	0.9264
SVM	Grid / Random	RBF, $C = 10$, $\gamma=\text{scale}$	0.9356

GridSearchCV and RandomizedSearchCV agreed on identical (or functionally identical) best parameters and CV accuracy for both models: Logistic Regression converged to $C = 100$ with L1 penalty and the liblinear solver (CV accuracy 0.9264 vs. 0.9261 for grid vs. random respectively – effectively tied), and SVM converged to the exact same configuration for both search methods (RBF, $C = 10$, $\gamma=\text{scale}$, CV accuracy 0.9356 for both). RandomizedSearchCV was noticeably faster for both models (Logistic Regression: 31.6 s vs. 58.6 s; SVM: 73.7 s vs. 83.3 s) while reaching the same quality solution.

8 Logistic Regression Performance

Metric	Value
Accuracy	0.9251
Precision	0.9250
Recall	0.9251
F1 Score	0.9248
Training Time (s)	0.847

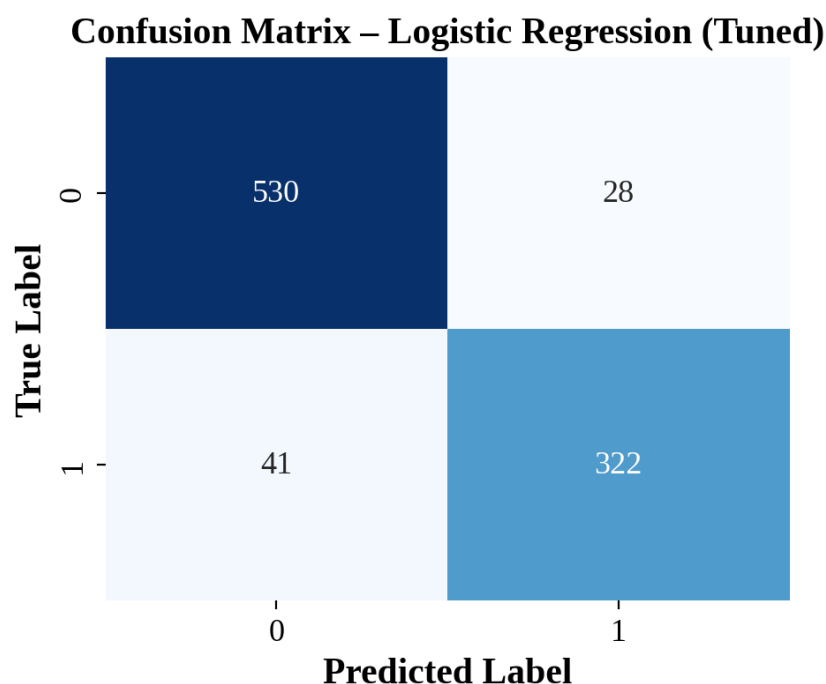


Figure 2: Confusion matrix for the tuned Logistic Regression model on the held-out test set.

Inference: The tuned model achieves balanced precision and recall, indicating it does not systematically favor one class over the other. Most residual errors are borderline emails whose word/character frequency profile sits between typical spam and ham patterns.

9 SVM Kernel-wise Performance

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9294	0.9093	0.387
Polynomial	0.7796	0.6220	0.348
RBF	0.9273	0.9055	0.186
Sigmoid	0.8849	0.8528	0.243

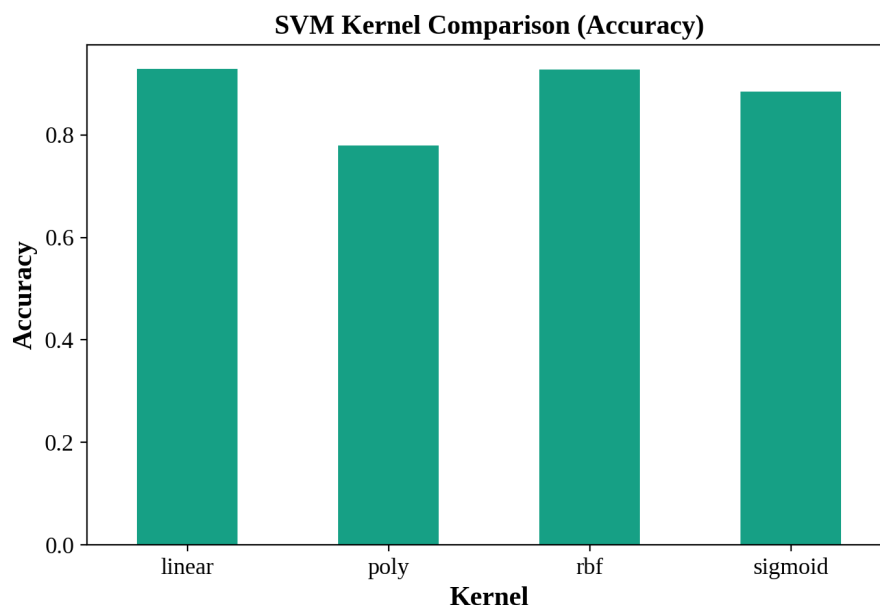


Figure 3: SVM kernel comparison (default hyperparameters) by test accuracy.

Inference: With default hyperparameters, the Linear and RBF kernels perform almost identically and far better than Sigmoid, and dramatically better than the (untuned) Polynomial kernel, whose default degree-3 polynomial expansion appears to distort the decision boundary badly on this 57-dimensional standardized feature space. This gap motivates the hyperparameter search in Section 6, which found that a properly tuned RBF kernel ($C = 10$) achieves the best cross-validated accuracy of all configurations tested.

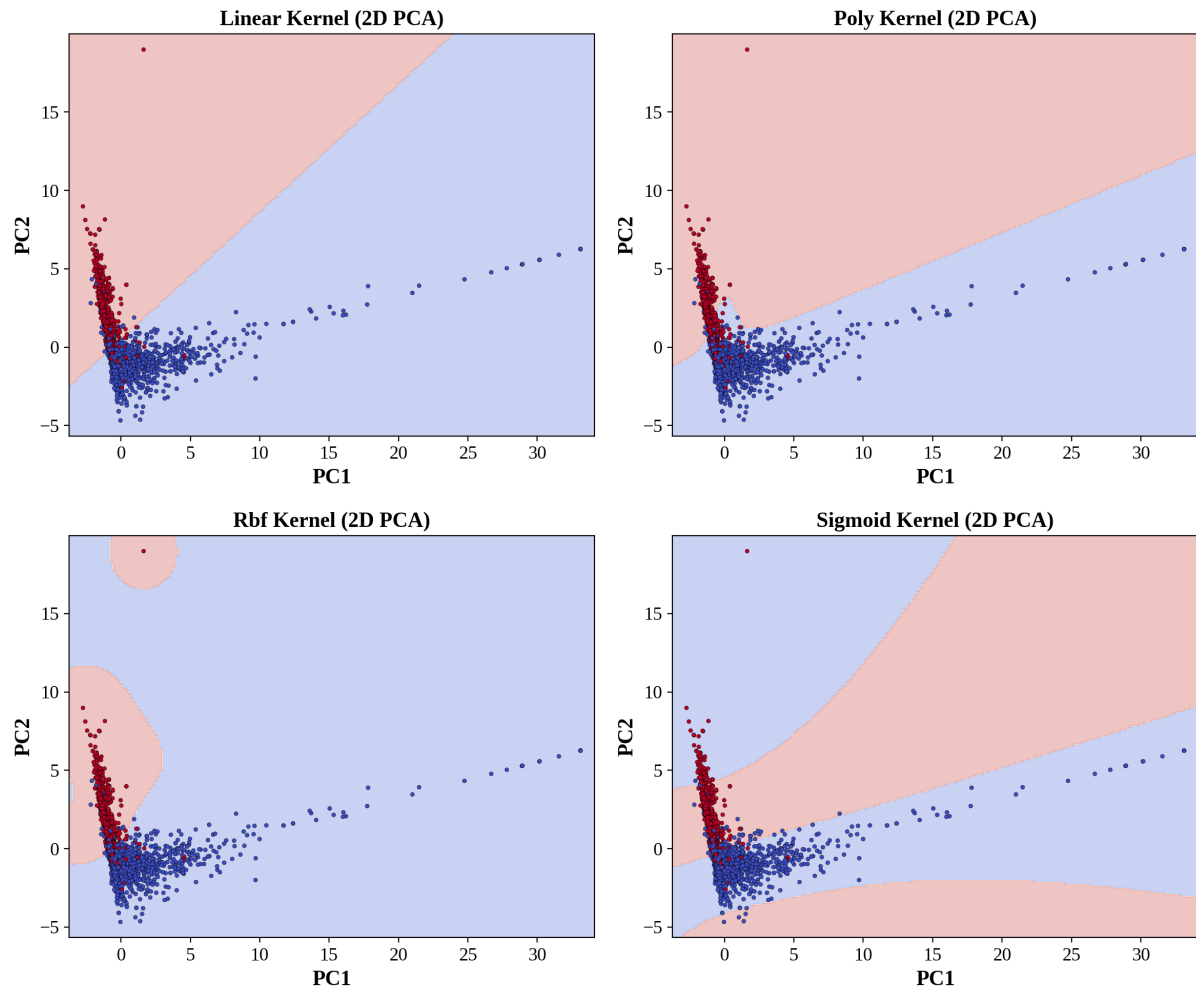


Figure 4: Decision boundaries of all four SVM kernels, visualized on a 2D PCA projection of the training data.

Inference: Projected onto the first two principal components, the Linear and Polynomial kernels produce very similar, nearly straight decision boundaries. The RBF kernel wraps a tighter, curved region around the dense ham cluster, reflecting its ability to model local, non-linear structure. The Sigmoid kernel produces a visibly distorted, non-convex boundary that misclassifies large regions of the spam cluster (bottom-left), which is consistent with its lower accuracy in Table 3 above – the sigmoid kernel is generally less numerically stable and more sensitive to hyperparameter choice than RBF on standardized data.

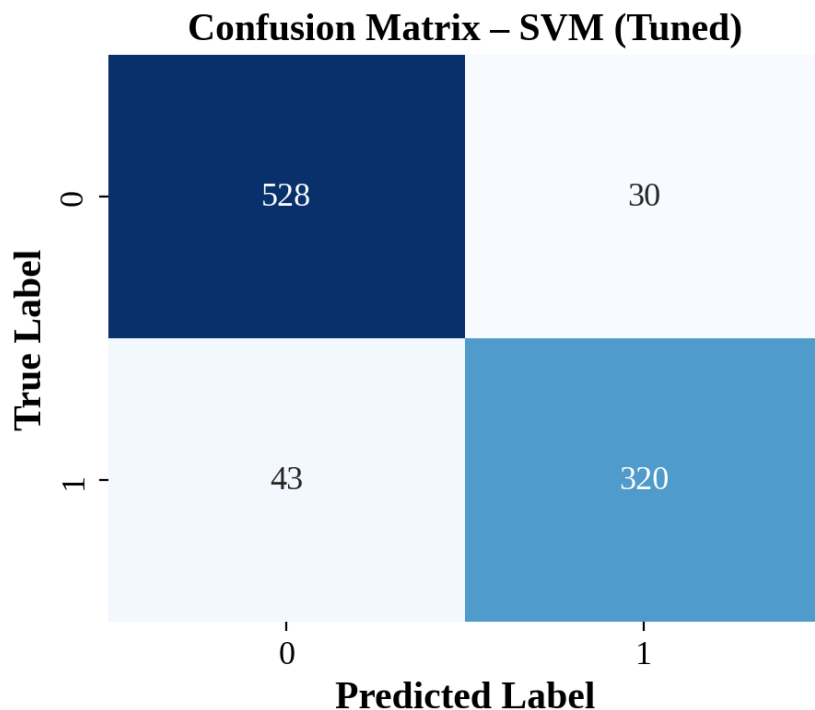


Figure 5: Confusion matrix for the tuned SVM (RBF, $C = 10$) model on the held-out test set.

Inference: The tuned SVM’s confusion matrix is similarly balanced to the Logistic Regression model’s, with a comparable overall error rate; the two models make similar types of errors, which is consistent with their near-identical ROC-AUC scores (Section 9).

10 K-Fold Cross-Validation Results ($K = 5$)

Fold	Logistic Regression	SVM
1	0.9121	0.9305
2	0.9359	0.9293
3	0.9272	0.9413
4	0.9283	0.9326
5	0.9261	0.9370
Average	0.9259	0.9341

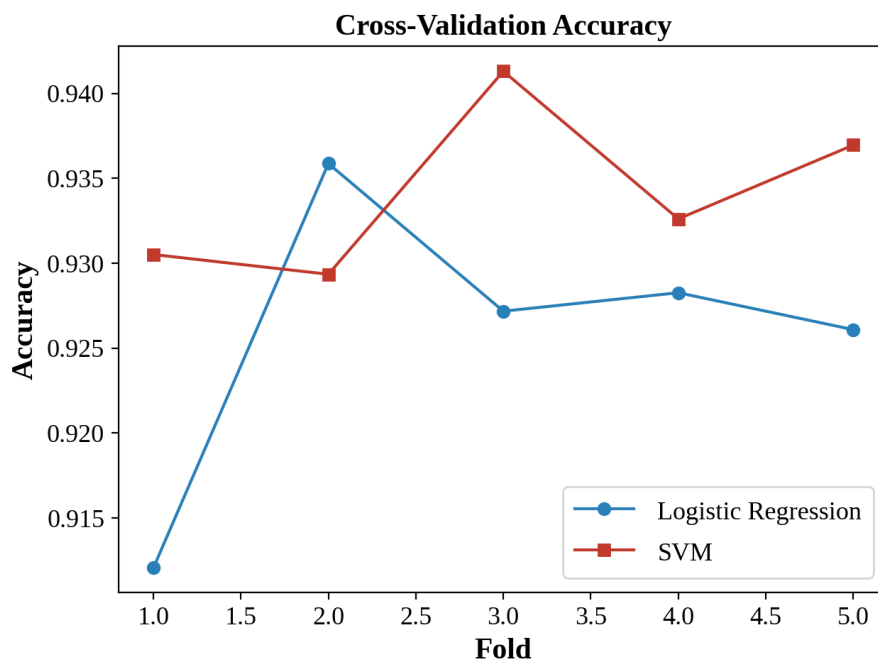


Figure 6: 5-fold cross-validation accuracy: Logistic Regression vs. SVM.

Inference: SVM's cross-validation accuracy is both higher on average (0.9341 vs. 0.9259) and comparably stable across folds relative to Logistic Regression, giving confidence that SVM's advantage in cross-validated accuracy is not an artifact of a single lucky fold.

11 Comparative Analysis

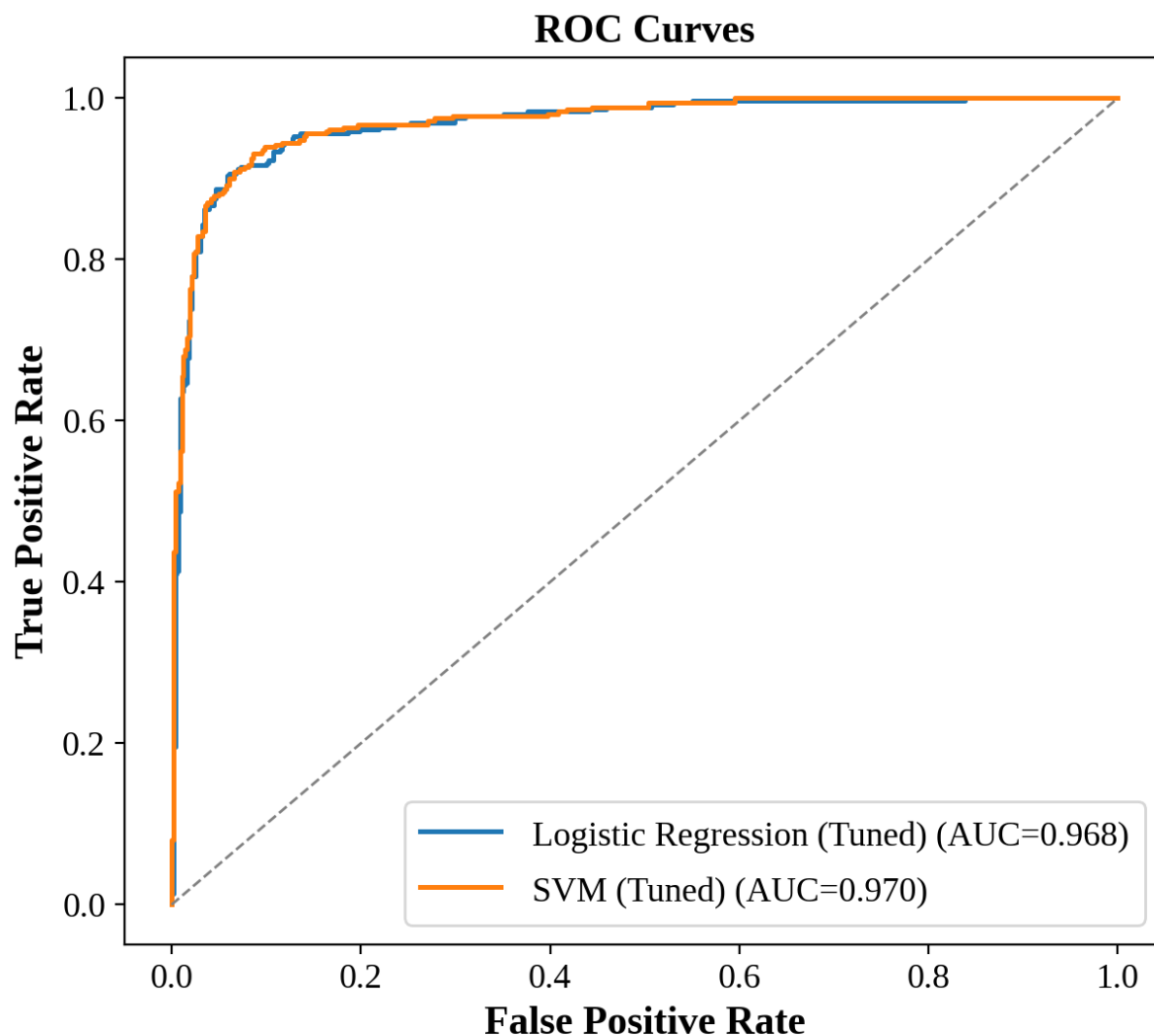


Figure 7: ROC curves for tuned Logistic Regression and tuned SVM.

Inference: The two models are nearly indistinguishable on ROC-AUC (Logistic Regression 0.9678 vs. SVM 0.9702), with SVM holding a very slight edge, particularly at low false-positive rates – a desirable property for a spam filter that must avoid misclassifying legitimate (ham) email.

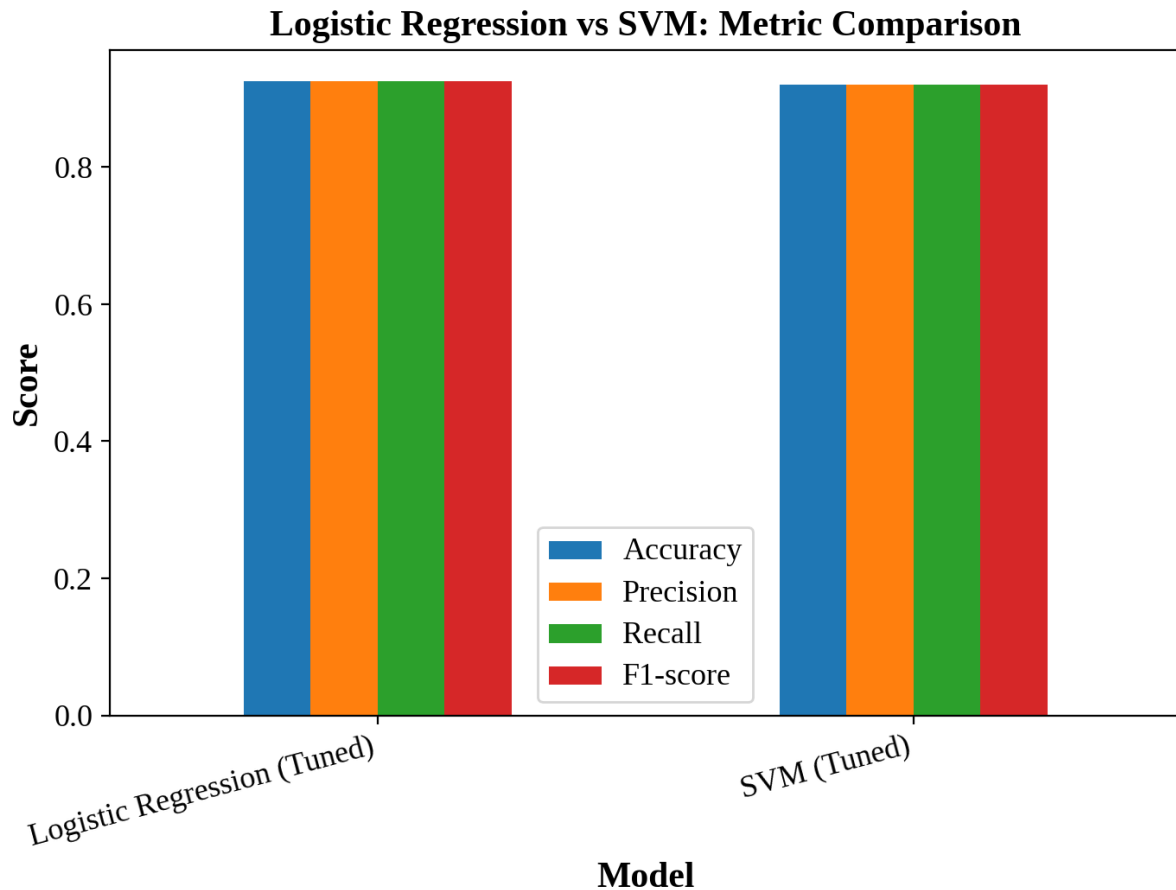


Figure 8: Logistic Regression vs. SVM: Accuracy, Precision, Recall, and F1 comparison.

Inference: On the single 80/20 test split, tuned Logistic Regression edges out tuned SVM on accuracy (0.9251 vs. 0.9207), precision, recall, and F1 by a small margin; however, the 5-fold cross-validated results (Section 7) show the opposite ranking, with SVM ahead on average accuracy. This apparent discrepancy is a useful illustration of why relying on a single train/test split can be misleading – the cross-validated numbers are the more trustworthy estimate of true generalization performance for both models.

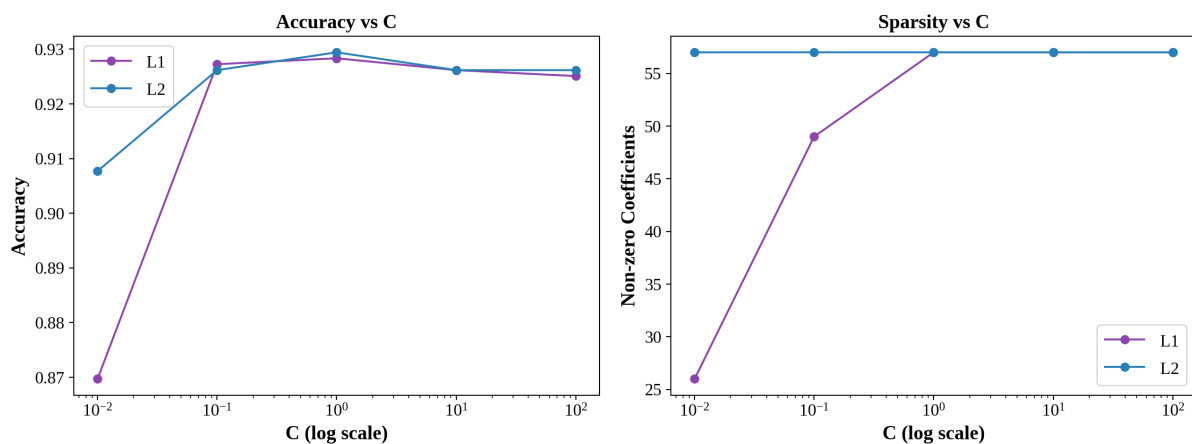


Figure 9: Left: Logistic Regression test accuracy vs. C , by penalty type. Right: number of non-zero coefficients vs. C (L1 sparsity effect).

Inference: Accuracy for both L1 and L2 peaks around $C = 1$ and degrades at the extremes – very small C (heavy regularization, $C = 0.01$) noticeably hurts L1 more than L2 (0.870 vs. 0.908 accuracy), since L1 is aggressively zeroing out features that still carry useful signal at that strength. The sparsity plot confirms this directly: at $C = 0.01$, L1 retains only 26 of 57 features, rising to the full 57 by $C = 1$ and beyond, while L2 never zeroes out any feature at any tested C . This is the clearest empirical demonstration in this experiment of L1’s feature-selection behavior versus L2’s pure shrinkage behavior.

Criterion	Logistic Regression	SVM
Accuracy (test split)	0.9251	0.9207
Accuracy (5-fold CV)	0.9259	0.9341
Model Complexity	Low	High
Training Time	Low (0.85 s)	Low–Moderate (0.99 s)
Interpretability	High	Low

12 Observations

- **Best-performing classifier:** By cross-validated accuracy (the more reliable estimate), the tuned **SVM with an RBF kernel** is the best-performing classifier (0.9341 vs. 0.9259), though the margin over tuned Logistic Regression is modest and the two models are statistically close on ROC-AUC (0.970 vs. 0.968).
- **Impact of regularization:** L1 regularization in Logistic Regression performs automatic feature selection, discarding up to 31 of 57 features at strong regularization ($C = 0.01$) with a real accuracy cost; L2 never discards a feature and is more robust at strong regularization. Both converge to near-identical, unregularization-level accuracy once $C \geq 1$, confirming that this dataset does not require heavy regularization to avoid overfitting – 4,601 samples is ample for a 57-feature linear model.
- **Kernel behavior in SVM:** Linear and RBF kernels are the clear winners; the polynomial kernel performs poorly with its default degree of 3 (likely overfitting to noise or producing numerically unstable feature expansions in 57 dimensions), and the sigmoid kernel’s boundary (Figure 4) is visibly less well-suited to this feature space’s geometry than RBF’s tighter, more localized boundary.
- **Bias–variance trade-off:** Logistic Regression, being a simpler linear model, exhibits comparatively higher bias, expressed here as its plateau in accuracy across a wide range of C values (Table in Section 9) rather than a sharp drop-off, indicating it is not overfitting even at low regularization. SVM’s RBF kernel, being more flexible, is more sensitive to hyperparameter choice (C and γ) – the untuned RBF SVM (default $C = 1$) already performed well (0.9273 accuracy), but tuning C up to 10 further reduced bias and improved cross-validated accuracy to 0.9356, illustrating the classic trade-off of higher model flexibility (lower bias, if properly tuned) against greater sensitivity to hyperparameter misspecification.

13 Conclusion

This experiment implemented and tuned Logistic Regression and SVM classifiers for spam detection on the Spambase dataset, evaluating four SVM kernels and comparing L1 vs. L2 regularization in Logistic Regression. GridSearchCV and RandomizedSearchCV converged on essentially identical optimal hyperparameters for both models (Logistic Regression: $C = 100$, L1; SVM: RBF kernel, $C = 10$), with RandomizedSearchCV consistently faster. Tuned Logistic Regression achieved marginally higher single-split test accuracy (0.9251), while tuned SVM (RBF) achieved higher and more reliable 5-fold cross-validated accuracy (0.9341) and a slightly higher ROC-AUC (0.9702). Given SVM's cross-validated advantage, it is the recommended classifier when raw predictive accuracy is the priority; Logistic Regression remains an attractive alternative when model interpretability (explicit, signed feature coefficients) or the lowest possible training cost is a priority, since it trains and predicts nearly as fast as SVM while remaining directly interpretable. Both models substantially outperform the untuned Polynomial and Sigmoid SVM kernels, reinforcing that kernel choice is at least as important as regularization strength when building an SVM-based spam classifier.

14 References

1. Scikit-learn Developers, “Logistic Regression,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
2. Scikit-learn Developers, “Support Vector Machines,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>
3. Scikit-learn Developers, “Tuning the hyper-parameters of an estimator,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html
4. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>